Database Management System

Unit-1

Introduction to Database Management System

DBMS (Database Management System)

- **Definition:**

- A database-management system (DBMS) is a collection of interrelated data and a set of programs to access those data.
- The collection of data, usually referred to as the database, contains information relevant to an enterprise. The primary goal of a DBMS is to provide a way to store and retrieve database information that is both convenient and efficient.

- A **Database Management System (DBMS)** is software that is used to **store, manage, retrieve, and manipulate data efficiently** in a database.

Database-System Applications

- Enterprise Information ◦
- Sales: For customer, product, and purchase information.
- ◦ Accounting: For payments, receipts, account balances, assets and other accounting information.
- ◦ Human resources: For information about employees, salaries, payroll taxes, and benefits, and for generation of paychecks.
- ◦ Manufacturing: For management of the supply chain and for tracking production of items in factories, inventories of items in warehouses and stores, and orders for items.
- Online retailers: For sales data noted above plus online order tracking, generation of recommendation lists, and maintenance of online product evaluations.

- Banking and Finance

- ◦ Banking: For customer information, accounts, loans, and banking transactions.
- ◦ Credit card transactions: For purchases on credit cards and generation of monthly statements.
- ◦ Finance: For storing information about holdings, sales, and purchases of financial instruments such as stocks and bonds; also for storing real-time market data to enable online trading by customers and automated trading by the firm

- Universities: For student information, course registrations, and grades (in addition to standard enterprise information such as human resources and accounting).
- Airlines: For reservations and schedule information. Airlines were among the first to use databases in a geographically distributed manner.
- Telecommunication: For keeping records of calls made, generating monthly bills, maintaining balances on prepaid calling cards, and storing information about the communication networks.

Functions of DBMS

- Data storage, retrieval, and update
- Data security and authorization
- Data integrity and consistency
- Backup and recovery
- Concurrency control
- Data sharing among multiple users

Advantages of DBMS

- Reduces data redundancy
- Improves data consistency
- Provides data security
- Supports data independence
- Enables multi-user access

Examples of DBMS

- MySQL
- Oracle
- SQL Server
- PostgreSQL
- MS Access

File Processing System

- **Definition:**
  A **File Processing System** is a traditional method of storing and managing data where **data is stored in separate files** and each application program manages its own data.

Characteristics of File Processing System

- Data is stored in individual files
- Each file is handled by a specific application
- No centralized control of data
- Programs and data are tightly coupled

Disadvantages of File Processing System

- **Data Redundancy** – Same data is stored in multiple files.
- **Data Inconsistency** – Updates in one file may not reflect in others.
- **Poor Data Security** – Limited access control.
- **Difficult Data Sharing** – Files cannot be easily shared among users.
- **Lack of Data Integrity** – No proper constraints to maintain accuracy.
- **No Backup and Recovery** – Manual and unreliable.
- **Program–Data Dependence** – Changes in file structure require program changes.

- **Examples**
- Student records stored in text files
- Payroll files maintained separately for each department

- **Advantages of DBMS over File Processing Systems**
- **Reduced Data Redundancy**
  DBMS stores data in a centralized database, avoiding unnecessary duplication of data that is common in file systems.
- **Improved Data Consistency**
  Since data is stored at one place, updates are reflected everywhere, maintaining consistency.
- **Better Data Security**
  DBMS provides authorization and access control, ensuring that only authorized users can access data.
- **Data Sharing**
  Multiple users can access and share data simultaneously without interference.
- **Data Integrity**
  Integrity constraints ensure accuracy and reliability of data.
- **Data Backup and Recovery**
  DBMS supports automatic backup and recovery in case of system failure.
- **Concurrency Control**
  Multiple users can work on the database at the same time without data conflicts.
- **Data Independence**
  Changes in data structure do not affect application programs.
- **Efficient Data Access**
  DBMS provides faster and more flexible data retrieval using queries.
- **Reduced Application Development Time**
  DBMS simplifies data management, reducing program complexity.

Data redundancy and inconsistency.

- Since different programmers create the files and application programs over a long period, the various files are likely to have different structures and the programs may be written in several programming languages. Moreover, the same information may be duplicated in several places (files).
- For example, if a student has a double major (say, music and mathematics) the address and telephone number of that student may appear in a file that consists of student records of students in the Music department and in a file that consists of student records of students in the Mathematics department
- This redundancy leads to higher storage and access cost. In addition, it may lead to data inconsistency; that is, the various copies of the same data may no longer agree.
- For example, a changed student address may be reflected in the Music department records but not elsewhere in the system.

- Difficulty in accessing data. The university clerk has now two choices: either obtain the list of all students and extract the needed information manually or ask a programmer to write the necessary application program. Both alternatives are obviously unsatisfactory.
- Data isolation. Because data are scattered in various files, and files may be in different formats, writing new application programs to retrieve the appropriate data is difficult.
- Integrity problems. The data values stored in the database must satisfy certain types of consistency constraints.

- Concurrent-access anomalies
- Consider department A, with an account balance of $10,000. If two department clerks debit the account balance (by say $500 and $100, respectively) of department A at almost exactly the same time, the result of the concurrent executions may leave the budget in an incorrect (or inconsistent) state.
- Security problems

Not every user of the database system should be able to access all the data.

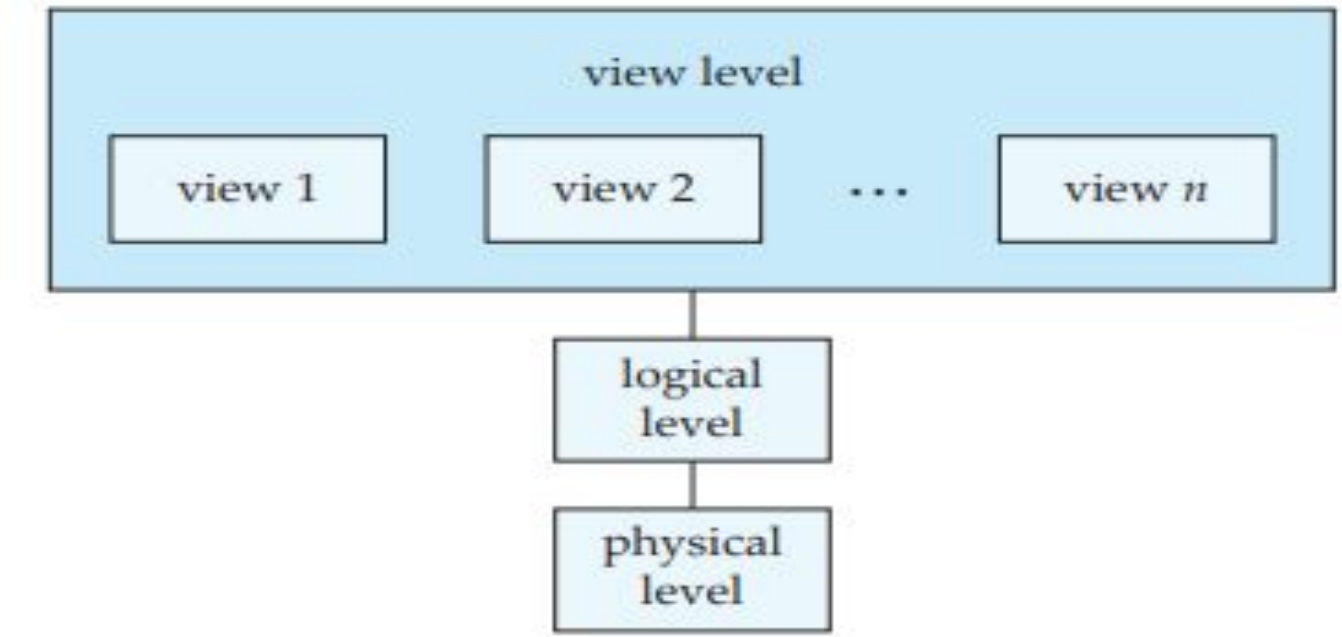
For example, in a university, payroll personnel need to see only that part of the database that has financial information. They do not need access to information about academic records

View of Data

- A database system is a collection of interrelated data and a set of programs that allow users to access and modify these data. A major purpose of a database system is to provide users with an abstract view of the data. That is, the system hides certain details of how the data are stored and maintained.
- Data Abstraction
- For the system to be usable, it must retrieve data efficiently

- Physical level. The lowest level of abstraction describes how the data are actually stored. The physical level describes complex low-level data structures in detail.
- Logical level. The next-higher level of abstraction describes what data are stored in the database, and what relationships exist among those data. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.
- View level. The highest level of abstraction describes only part of the entire database. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system. The system may provide many views for the same database.

The three levels of data abstraction.



Example

```
type instructor = record
                    ID : char (5);
                    name : char (20);
                    dept_name : char (20);
                    salary : numeric (8,2);
                  end;
```

- *department*, with fields *dept_name*, *building*, and *budget*
- *course*, with fields *course_id*, *title*, *dept_name*, and *credits*
- *student*, with fields *ID*, *name*, *dept_name*, and *tot_cred*

At the physical level, an instructor, department, or student record can be described as a block of consecutive storage locations. The compiler hides this level of detail from programmers.

At the logical level, each such record is described by a type definition, as in the previous code segment, and the interrelationship of these record types is defined as well.

At the view level, computer users see a set of application programs that hide details of the data types. At the view level, several views of the database are defined, and a database user sees some or all of these views.

Instances and Schemas

- Databases change over time as information is inserted and deleted. The collection of information stored in the database at a particular moment is called an instance of the database.
- The overall design of the database is called the database schema. Schemas are changed infrequently, if at all.
- Database systems have several schemas, partitioned according to the levels of abstraction. The physical schema describes the database design at the physical level, while the logical schema describes the database design at the logical level. A database may also have several schemas at the view level, sometimes called subschemas, that describe different views of the database

- Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. This is referred to as **physical data independence.**

Data Models

- Underlying the structure of a database is the data model: a collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints. A data model provides a way to describe the design of a database at the physical, logical, and view levels.

The relational model

- The relational model uses a collection of tables to represent both data and the relationships among those data. Each table has multiple columns, and each column has a unique name. Tables are also known as relations.

Entity-Relationship Model

- The entity-relationship (E-R) data model uses a collection of basic objects, called entities, and relationships among these objects. An entity is a “thing” or “object” in the real world that is distinguishable from other objects.

Object-Based Data Model.

- Object-oriented programming (especially in Java, C++, or C#) has become the dominant software-development methodology. This led to the development of an object-oriented data model that can be seen as extending the E-R model with notions of encapsulation, methods (functions), and object identity.

Semi-structured Data Model.

- Semi-structured data model permits the specification of data where individual data items of the same type may have different sets of attributes.
- The Extensible Markup Language (XML) is widely used to represent semi-structured data

Database Languages

- A database system provides a
- data-definition language to specify the database schema and
- data-manipulation language to express database queries and updates.
- definition and data-manipulation languages are not two separate languages; instead they simply form parts of a single database language, such as the widely used SQL language.

Data-Manipulation Language

- Data-manipulation language (DML) is a language that enables users to access or manipulate data as organized by the appropriate data model. The types of access are:
- Retrieval of information stored in the database
- Insertion of new information into the database
- Deletion of information from the database
- Modification of information stored in the database

- There are basically two types:
- **Procedural DMLs** require a user to specify *what* data are needed and *how* to get those data.
- **Declarative DMLs** (also referred to as **nonprocedural DMLs**) require a user to specify *what* data are needed *without* specifying how to get those data.

- A **query** is a statement requesting the retrieval of information. The portion of a DML that involves information retrieval is called a **query language**.

Data-Definition Language

- We specify a database schema by a set of definitions expressed by a special language called a data-definition language (DDL). The DDL is also used to specify additional properties of the data.
- We specify the storage structure and access methods used by the database system by a set of statements in a special type of DDL called a **data storage and definition** language.
- These statements define the implementation details of the database schemas, which are usually hidden from the users.

DDL

- The data values stored in the database must satisfy certain consistency constraints
- For example, suppose the university requires that the account balance of a department must never be negative. The DDL provides facilities to specify such constraints. The database system checks these constraints every time the database is updated

- Domain Constraints. A domain of possible values must be associated with every attribute (for example, integer types, character types, date/time types).Declaring an attribute to be of a particular domain acts as a constraint on the values that it can take.
- Referential Integrity. There are cases where we wish to ensure that a value that appears in one relation for a given set of attributes also appears in a certain set of attributes in another relation (referential integrity).
- Assertions. An assertion is any condition that the database must always satisfy. Domain constraints and referential-integrity constraints are special forms of assertions.
- For example, “Every department must have at least five courses offered every semester” must be expressed as an assertion.

- **Authorization**. We may want to differentiate among the users as far as the type of access they are permitted on various data values in the database. These differentiations are expressed in terms of **authorization**, the most common being:
- **read authorization**, which allows reading, but not modification, of data;
- **insert authorization**, which allows insertion of new data, but not modification of existing data;
- **update authorization**, which allows modification, but not deletion, of data; and
- **delete authorization**, which allows deletion of data.

Important Note

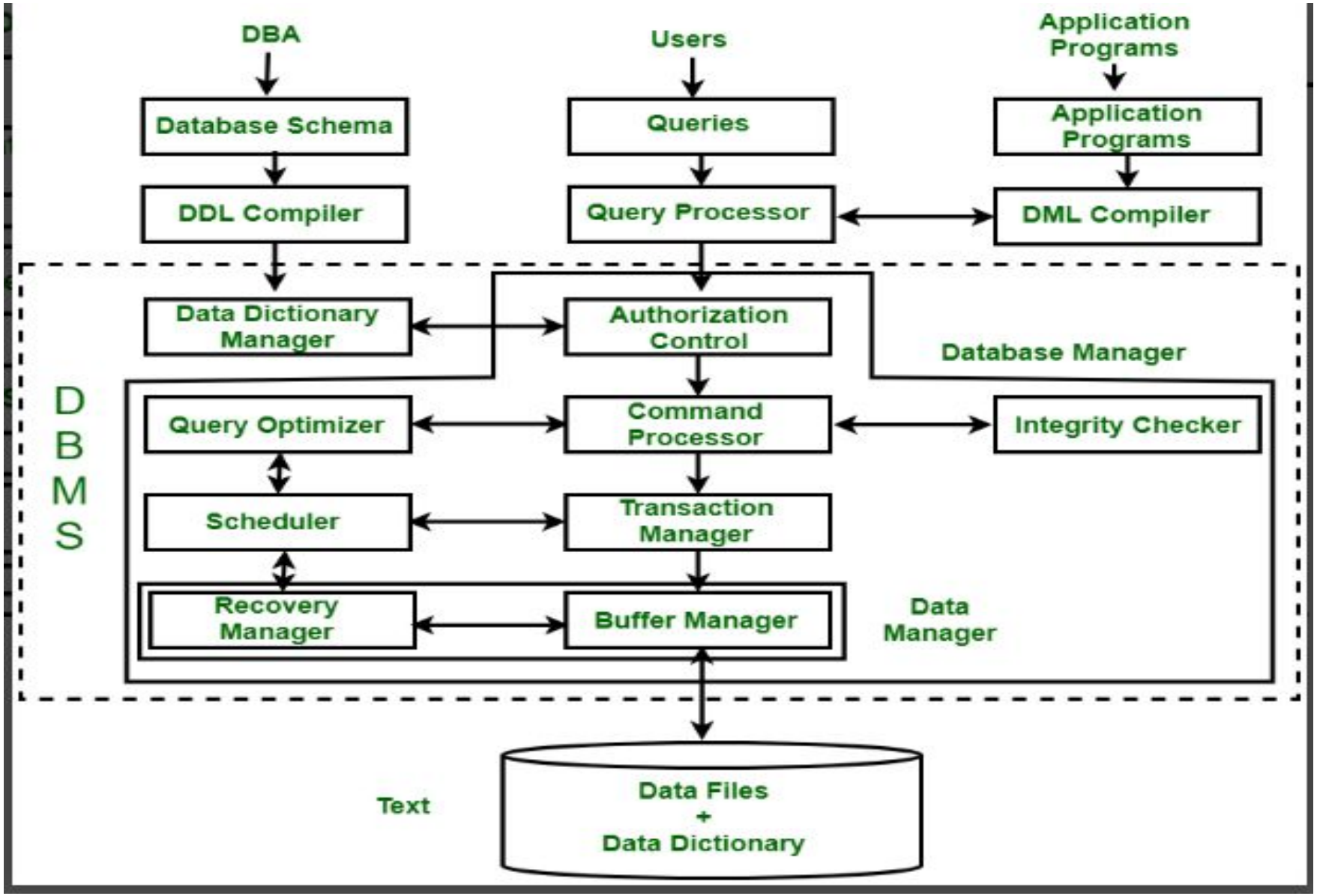
- The output of the DDL is placed in the **data dictionary**, which contains **metadata**—that is, data about data.
- The data dictionary is considered to be a special type of table that can only be accessed and updated by the database system itself (not a regular user).
- The database system consults the data dictionary before reading or modifying actual data.

Database Architecture vs. Tier Architecture

- Structure of Database Management System is also referred to as Overall System Structure or Database Architecture but it is different from the Tier architecture of Database.

- Database Architecture refers to the internal components of the DBMS, including the Query Processor, Storage Manager, and Disk Storage. It also defines the interaction of these components.

- Tier Architecture typically refers to the multi-layered setup in an application where DBMS serves as the data layer, but it is distinct from Database Architecture, which refers to the internal structure and levels (internal, conceptual, and external) of the DBMS.


DBA
Users
Application Programs
Database Schema
Queries
Application Programs
DDL Compiler
Query Processor
DML Compiler
Data Dictionary Manager
Authorization Control
Database Manager
DBMS
Query Optimizer
Command Processor
Integrity Checker
Scheduler
Transaction Manager
Recovery Manager
Buffer Manager
Data Manager
Text
Data Files + Data Dictionary

Query Processor

- It interprets the requests (queries) received from end user via an application program into instructions. It also executes the user request which is received from the DML compiler. Query Processor contains the following components -

- DML Compiler: It processes the DML statements into low level instruction (machine language), so that they can be executed.
- DDL Interpreter: It processes the DDL statements into a set of table containing meta data (data about data).
- Embedded DML Pre-compiler: It processes DML statements embedded in an application program into procedural calls.
- Query Optimizer: The Query Optimizer executes instructions generated by the DML Compiler and improves query execution efficiency by choosing the best query plan, considering factors such as indexing, join order, and available system resources. For instance, if a query involves joining two large tables, the optimizer will select the best join order to minimize query execution time.

Storage Manager

- Storage Manager is an interface between the data stored in the database and the queries received. It is also known as Database Control System. It maintains the consistency and integrity of the database by applying the constraints and executing the DCL statements. It is responsible for updating, storing, deleting, and retrieving data in the database. It contains the following components:

- Authorization Manager: It ensures role-based access control, i.e,. checks whether the particular person is privileged to perform the requested operation or not.

- Integrity Manager: It checks the integrity constraints when the database is modified.
- Transaction Manager: It controls concurrent access by performing the operations in a scheduled way that it receives the transaction. Thus, it ensures that the database remains in the consistent state before and after the execution of a transaction.

- File Manager: It manages the file space and the data structure used to represent information in the database.

- Buffer Manager: It is responsible for cache memory and the transfer of data between the secondary storage and main memory.

Disk Storage

- It contains the following essential components:

- Data Files: It stores the actual data in the database.
- Data Dictionary: It contains the information about the structure of database objects such as tables, constraints, and relationships. It is the repository of information that governs the metadata.
- Indices: Provides faster data retrieval by allowing the DBMS to find records quickly, improving query performance.

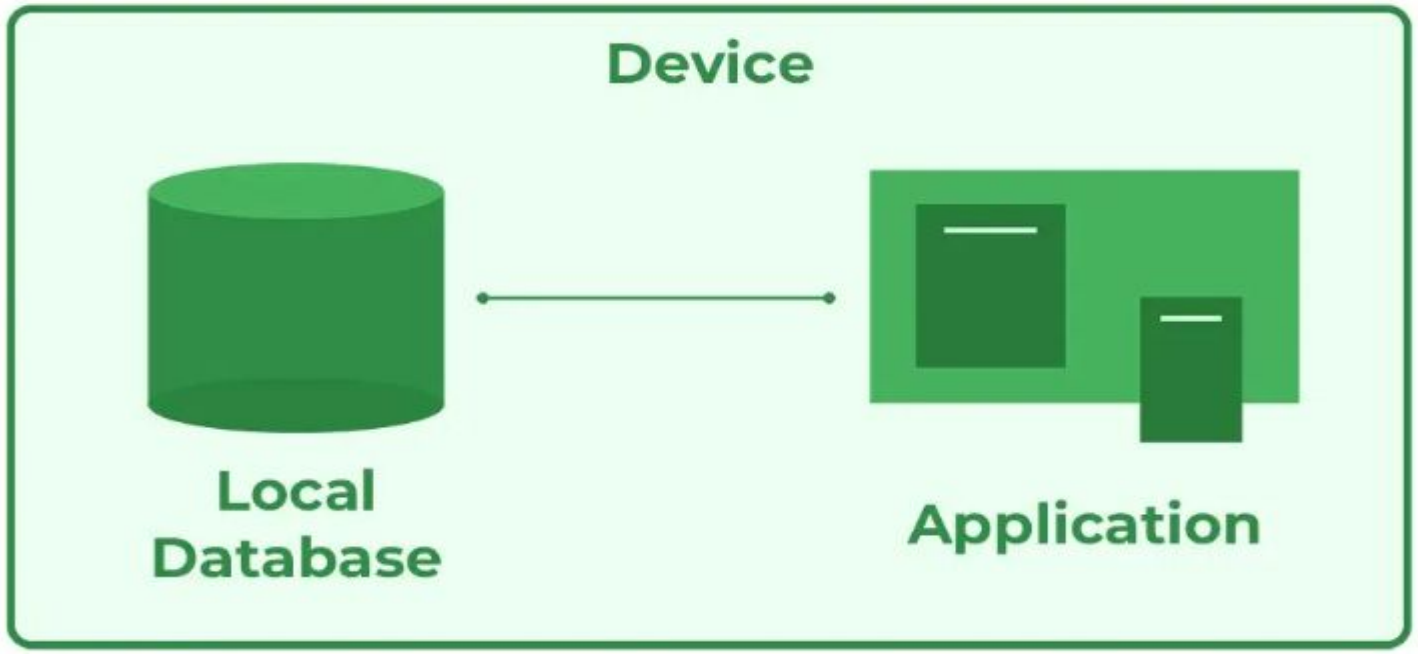
Types of DBMS Architecture

- There are several types of DBMS Architecture that we use according to the usage requirements.

- 1-Tier Architecture
- 2-Tier Architecture
- 3-Tier Architecture

1-Tier Architecture

- In 1-Tier Architecture, the user works directly with the database on the same system. This means the client, server and database are all in one application. The user can open the application, interact with the data and perform tasks without needing a separate server or network connection.



DBMS 1-Tier Architecture

- A common example is Microsoft Excel. Everything from the user interface to the logic and data storage happens on the same device. The user enters data, performs calculations and saves files directly on their computer.
- This setup is simple and easy to use, making it ideal for personal or standalone applications. It does not require a network or complex setup, which is why it's often used in small-scale or individual use cases.
- This architecture is simple and works well for personal, standalone applications where no external server or network connection is needed.

Advantages of 1-Tier Architecture

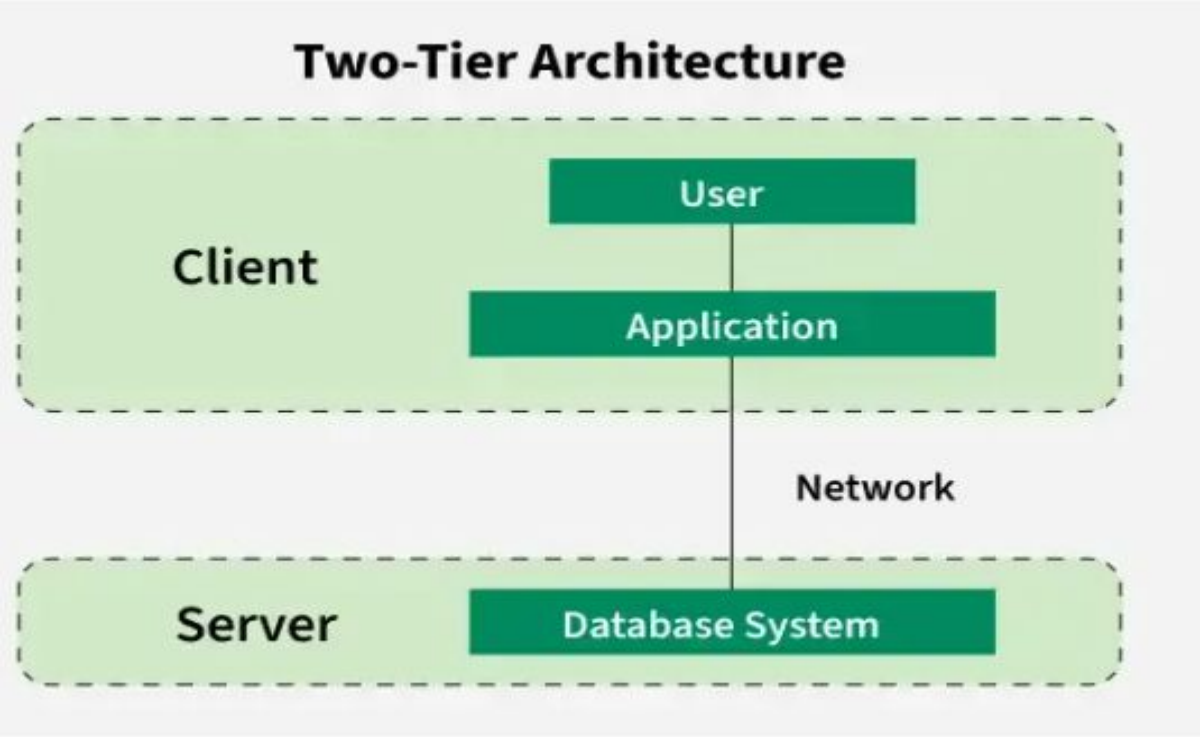
- Simple Architecture: 1-Tier Architecture is the most simple architecture to set up, as only a single machine is required to maintain it.
- Cost-Effective: No additional hardware is required for implementing 1-Tier Architecture, which makes it cost-effective.
- Easy to Implement: 1-Tier Architecture can be easily deployed and hence it is mostly used in small projects.

Disadvantages of 1-Tier Architecture

- Limited to Single User: Only one person can use the application at a time. It’s not designed for multiple users or teamwork.
- Poor Security: Since everything is on the same machine, if someone gets access to the system, they can access both the data and the application easily.
- No Centralized Control: Data is stored locally, so there's no central database. This makes it hard to manage or back up data across multiple devices.
- Hard to Share Data: Sharing data between users is difficult because everything is stored on one computer.

2-Tier Architecture

- The 2-tier architecture is similar to a basic client-server model. The application at the client end directly communicates with the database on the server side. APIs like ODBC and JDBC are used for this interaction. The server side is responsible for providing query processing and transaction management functionalities.



DBMS 2-Tier Architecture

- For Example: A Library Management System used in schools or small organizations is a classic example of two-tier architecture.
- Client Layer (Tier 1): This is the user interface that library staff or users interact with. For example they might use a desktop application to search for books, issue them, or check due dates.
- Database Layer (Tier 2): The database server stores all the library records such as book details, user information and transaction logs.
- The client layer sends a request (like searching for a book) to the database layer which processes it and sends back the result. This separation allows the client to focus on the user interface, while the server handles data storage and retrieval.

Advantages of 2-Tier Architecture

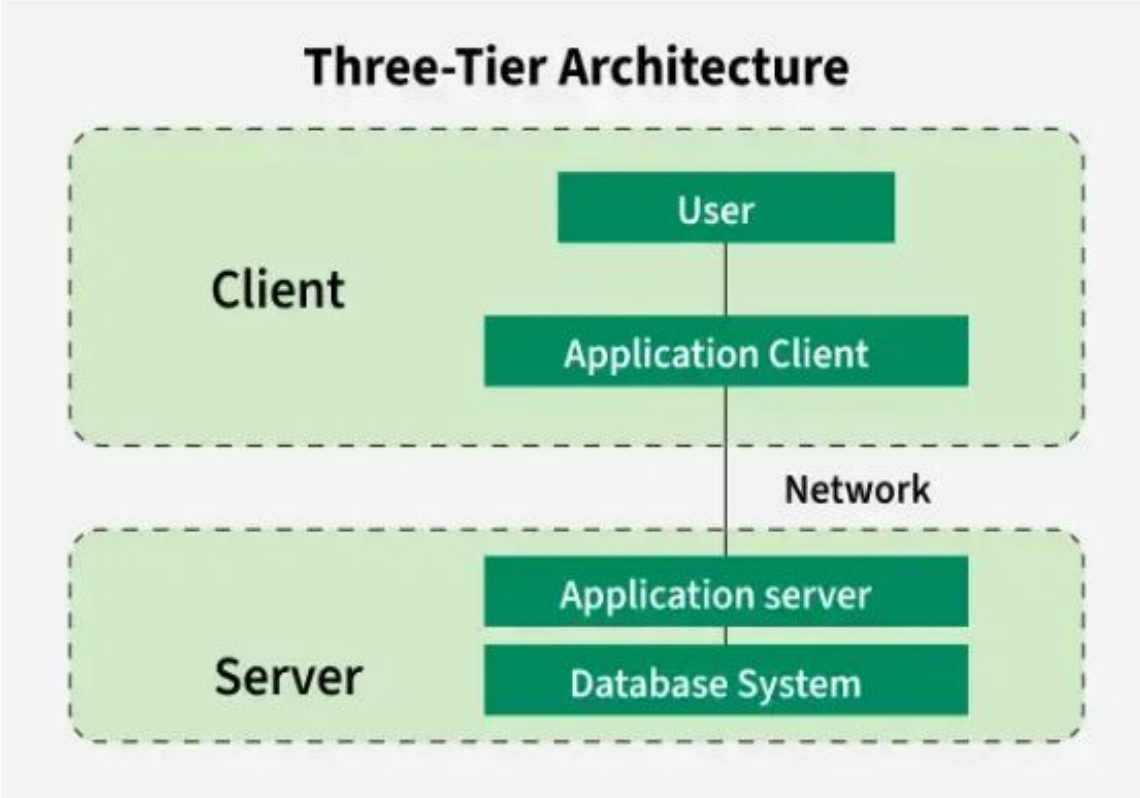
- Easy to Access: 2-Tier Architecture makes easy access to the database, which makes fast retrieval.
- Scalable: We can scale the database easily, by adding clients or upgrading hardware.
- Low Cost: 2-Tier Architecture is cheaper than 3-Tier Architecture and Multi-Tier Architecture.
- Easy Deployment: 2-Tier Architecture is easier to deploy than 3-Tier Architecture.
- Simple: 2-Tier Architecture is easily understandable as well as simple because of only two components.

Disadvantages of 2-Tier Architecture

- Limited Scalability: As the number of users increases, the system performance can slow down because the server gets overloaded with too many requests.
- Security Issues: Clients connect directly to the database, which can make the system more vulnerable to attacks or data leaks.
- Tight Coupling: The client and the server are closely linked. If the database changes, the client application often needs to be updated too.
- Difficult Maintenance: Managing updates, fixing bugs, or adding features becomes harder when the number of users or systems increases.

3-Tier Architecture

- In 3-Tier Architecture, there is another layer between the client and the server. The client does not directly communicate with the server. Instead, it interacts with an application server which further communicates with the database system and then the query processing and transaction management takes place. This intermediate layer acts as a medium for the exchange of partially processed data between the server and the client. This type of architecture is used in the case of large web applications.



DBMS 3-Tier Architecture

- Example: E-commerce Store

- User: You visit an online store, search for a product and add it to your cart.
- Processing: The system checks if the product is in stock, calculates the total price and applies any discounts.
- Database: The product details, your cart and order history are stored in the database for future reference.

Advantages of 3-Tier Architecture

- Enhanced scalability: Scalability is enhanced due to the distributed deployment of application servers. Now, individual connections need not be made between the client and server.
- Data Integrity: 3-Tier Architecture maintains Data Integrity. Since there is a middle layer between the client and the server, data corruption can be avoided/removed.
- Security: 3-Tier Architecture Improves Security. This type of model prevents direct interaction of the client with the server thereby reducing access to unauthorized data.

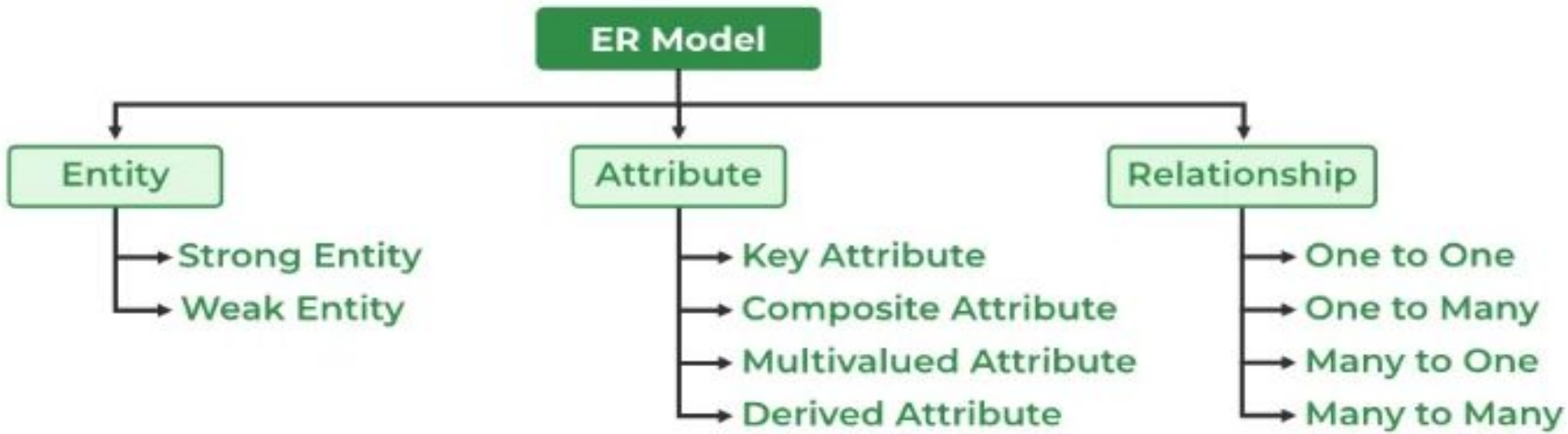
Disadvantages of 3-Tier Architecture

- More Complex: 3-Tier Architecture is more complex in comparison to 2-Tier Architecture. Communication Points are also doubled in 3-Tier Architecture.
- Difficult to Interact: It becomes difficult for this sort of interaction to take place due to the presence of middle layers.
- Slower Response Time: Since the request passes through an extra layer (application server), it may take more time to get a response compared to 2-Tier systems.
- Higher Cost: Setting up and maintaining three separate layers (client, server and database) requires more hardware, software and skilled people. This makes it more expensive.

Introduction of ER Model

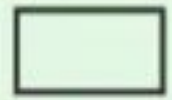
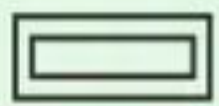
- The Entity-Relationship Model (ER Model) is a conceptual model for designing a database. This model represents the logical structure of a database, including entities, their attributes, and relationships between them.

- Entity: An object that is stored as data. E.g: Student, Course, or Company.
- Attribute: Properties that describe an entity. E.g: StudentID, CourseName, or EmployeeEmail.
- Relationship: A connection between entities. E.g: Student enrolls in a Course.


ER Model
Entity
Strong Entity
Weak Entity
Attribute
Key Attribute
Composite Attribute
Multivalued Attribute
Derived Attribute
Relationship
One to One
One to Many
Many to One
Many to Many

Use ER Diagrams In DBMS

- ER diagrams represent the E-R model in a database, making them easy to convert into relations (tables).
- These diagrams serve the purpose of real-world modeling of objects which makes them intently useful.
- Unlike technical schemas, ER diagrams require no technical knowledge of the underlying DBMS used.
- They visually model data and its relationships, making complex systems easier to understand.

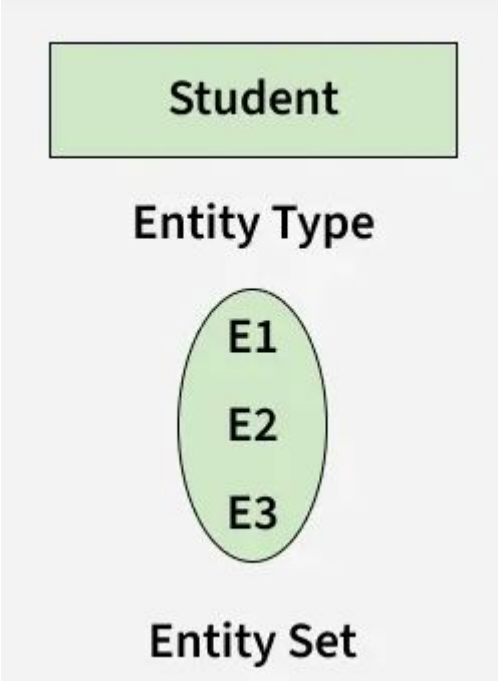
Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

- Entity
- An Entity represents a real-world object, concept or thing about which data is stored in a database. It act as a building block of a database. Tables in relational database represent these entities.

- Example of entities:

- Real-World Objects: Person, Car, Employee etc.
- Concepts: Course, Event, Reservation etc.
- Things: Product, Document, Device etc.

- Entity Set
- An entity refers to an individual object of an entity type, and the collection of all entities of a particular type is called an entity set. For example, E1 is an entity that belongs to the entity type "Student," and the group of all students forms the entity set.
- In the ER diagram below, the entity type is represented as:



Types of Entity

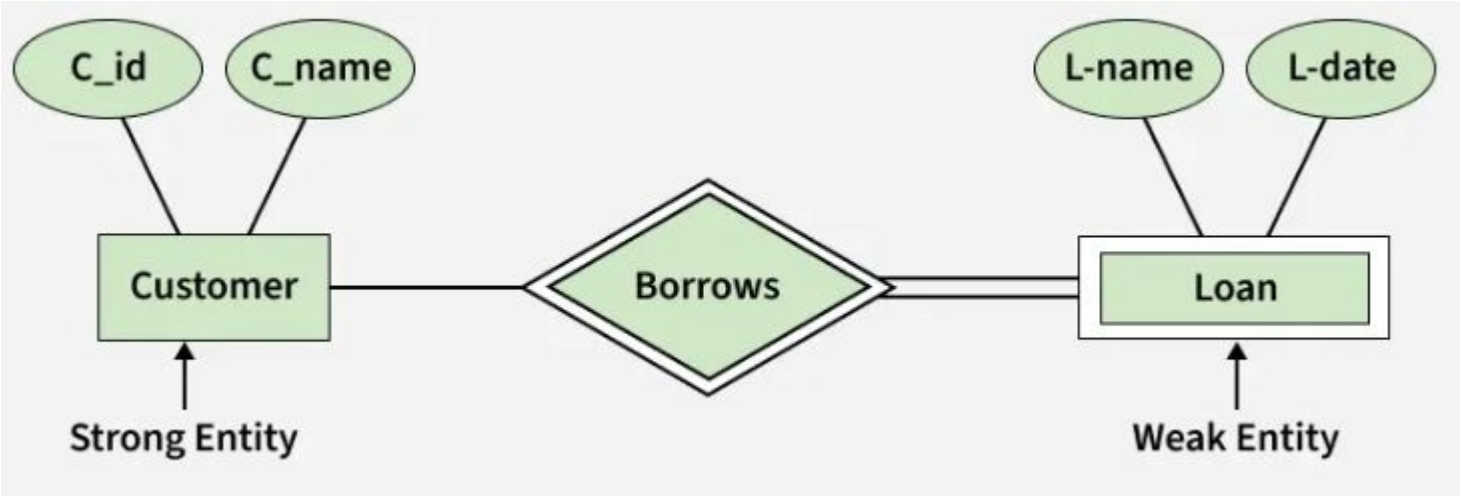
- There are two main types of entities:

- 1. Strong Entity
- A Strong Entity is a type of entity that has a key Attribute that can uniquely identify each instance of the entity. A Strong Entity does not depend on any other Entity in the Schema for its identification. It has a primary key that ensures its uniqueness and is represented by a rectangle in an ER diagram.

- 2. Weak Entity
- A Weak Entity cannot be uniquely identified by its own attributes alone. It depends on a strong entity to be identified. A weak entity is associated with an identifying entity (strong entity), which helps in its identification. A weak entity are represented by a double rectangle. The participation of weak entity types is always total. The relationship between the weak entity type and its identifying strong entity type is called identifying relationship and it is represented by a double diamond.

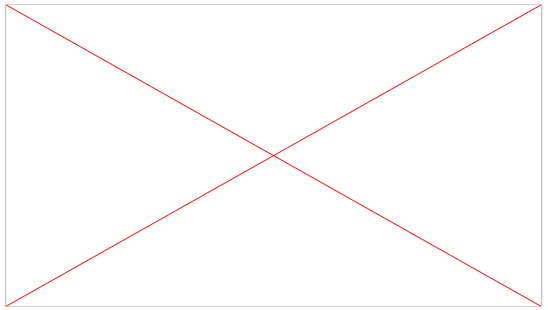
Example:

- A company may store the information of dependents (Parents, Children, Spouse) of an Employee. But the dependents can't exist without the employee. So dependent will be a Weak Entity Type and Employee will be identifying entity type for dependent, which means it is Strong Entity Type.



Attributes in ER Model

Attributes are the properties that define the entity type. For example, for a Student entity Roll_No, Name, DOB, Age, Address, and Mobile_No are the attributes that define entity type Student. In ER diagram, the attribute is represented by an oval.



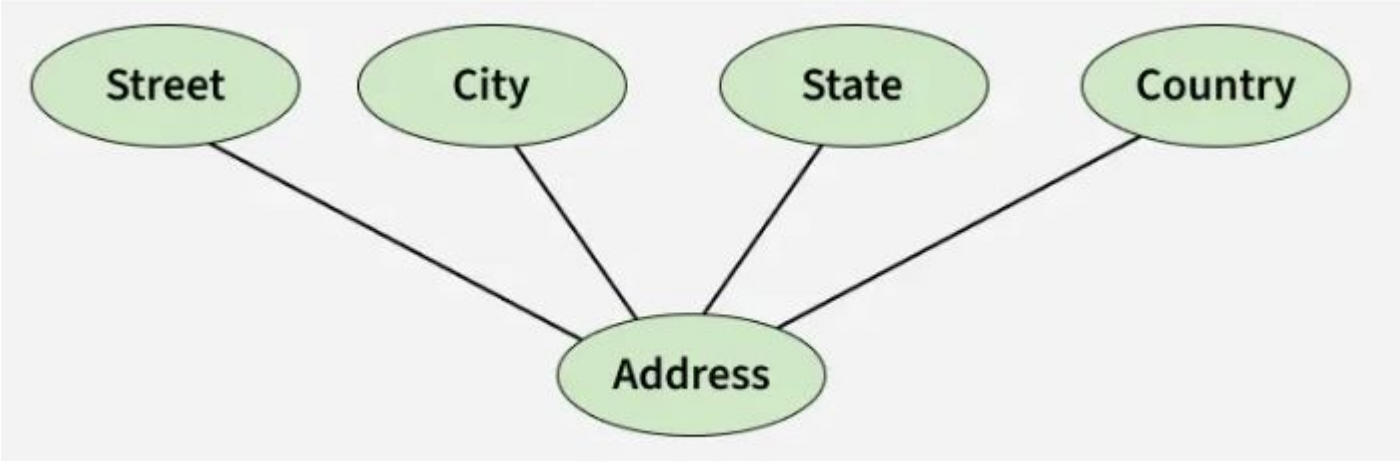
Types of Attributes

- 1. Key Attribute
- The attribute which uniquely identifies each entity in the entity set is called the key attribute. For example, Roll_No will be unique for each student. In ER diagram, the key attribute is represented by an oval with an underline.

Key Attribute

Composite Attribute

- An attribute composed of many other attributes is called a composite attribute. For example, the Address attribute of the student Entity type consists of Street, City, State, and Country. In ER diagram, the composite attribute is represented by an oval comprising of ovals.



Composite Attribute

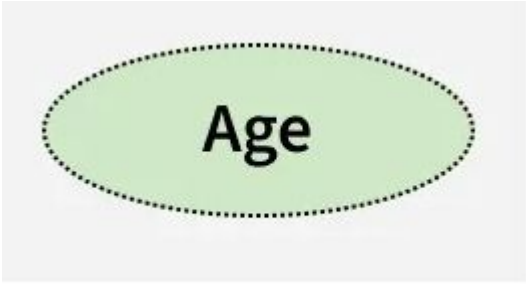
Multivalued Attribute

- An attribute consisting of more than one value for a given entity. For example, Phone_No (can be more than one for a given student). In ER diagram, a multivalued attribute is represented by a double oval.

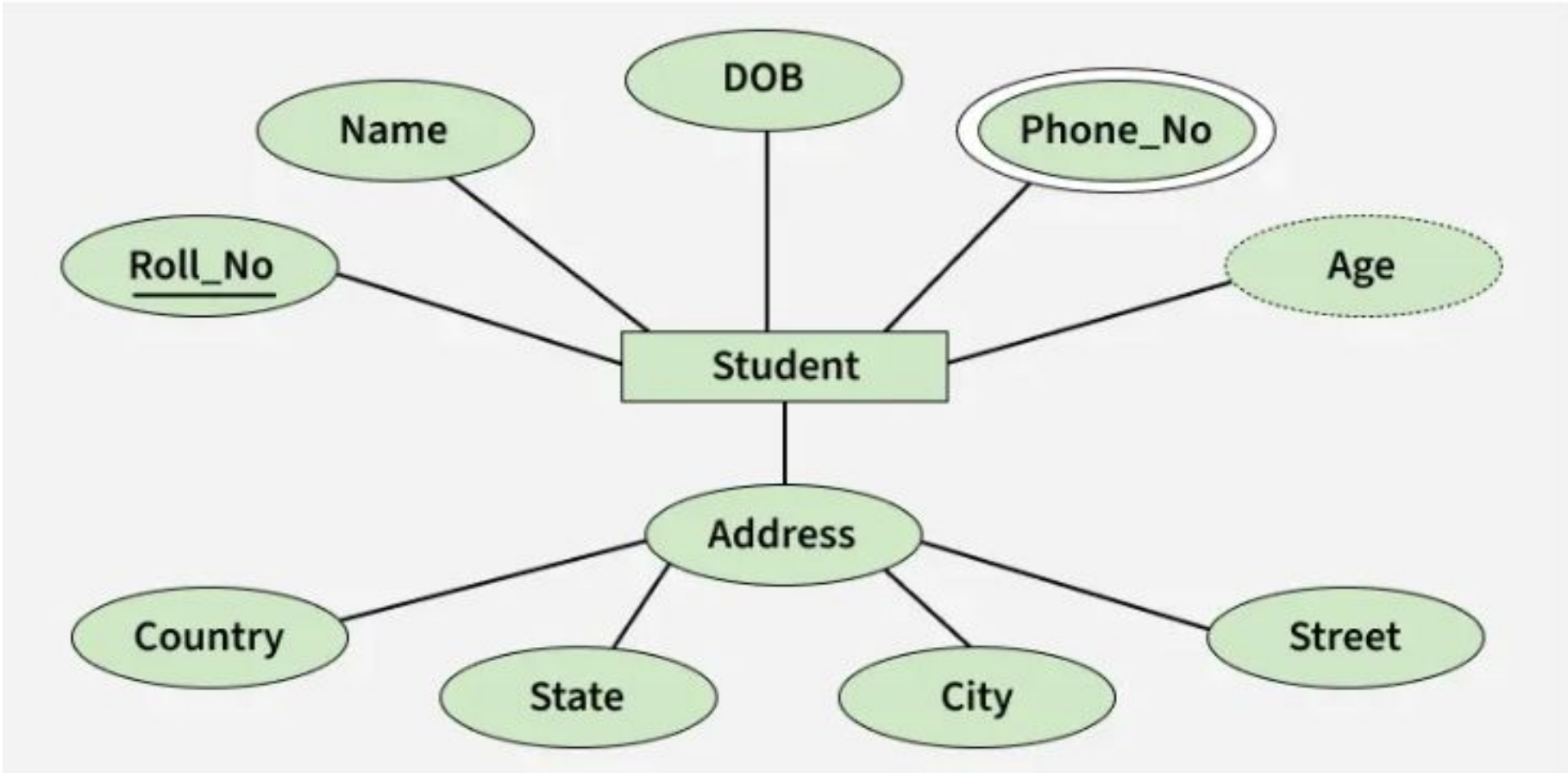
Multivalued Attribute

Derived Attribute

- An attribute that can be derived from other attributes of the entity type is known as a derived attribute. e.g.; Age (can be derived from DOB). In ER diagram, the derived attribute is represented by a dashed oval.

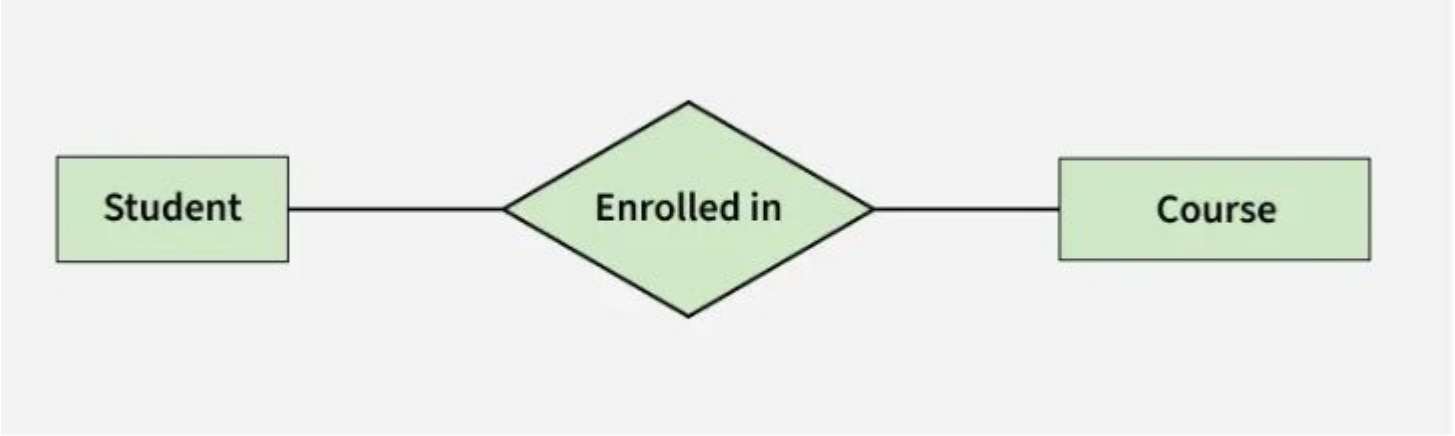


Derived Attribute


DOB
Name
Phone_No
Roll_No
Age
Student
Address
Country
State
City
Street

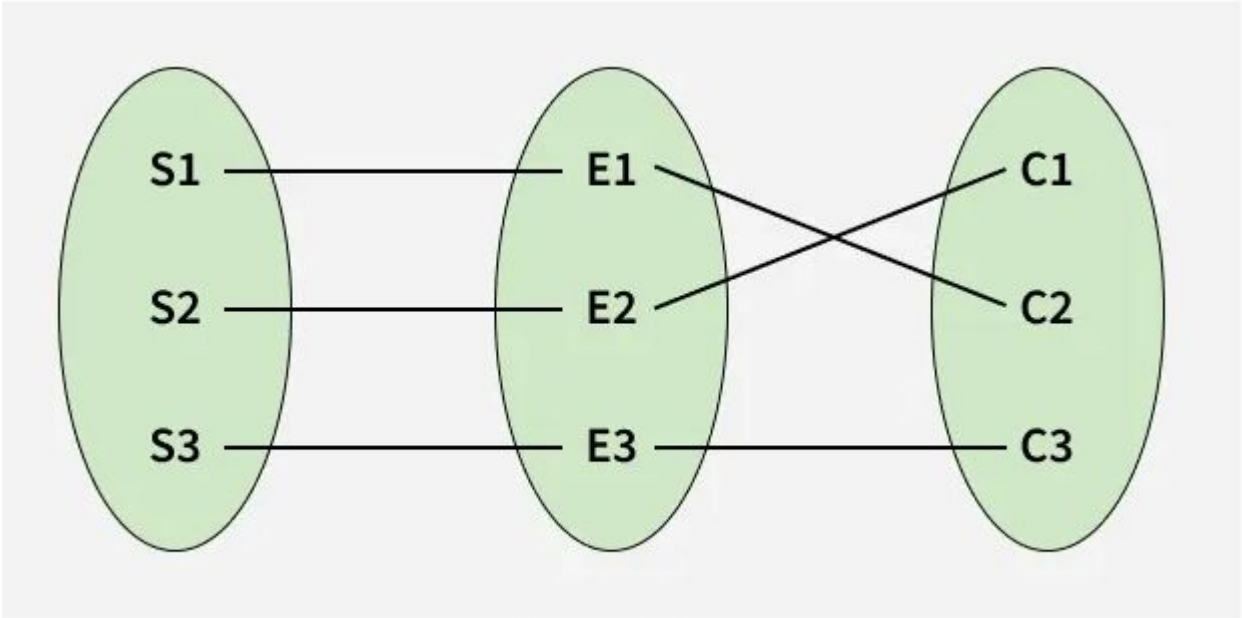
Relationship Type and Relationship Set

- A Relationship Type represents the association between entity types. For example, ‘Enrolled in’ is a relationship type that exists between entity type Student and Course. In ER diagram, the relationship type is represented by a diamond and connecting the entities with lines.



Entity-Relationship Set

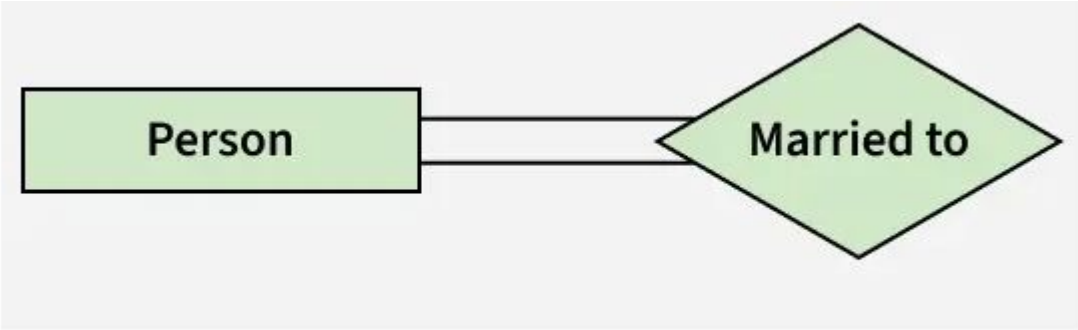
- A set of relationships of the same type is known as a relationship set. The following relationship set depicts S1 as enrolled in C2, S2 as enrolled in C1, and S3 as registered in C3.



Relationship Set

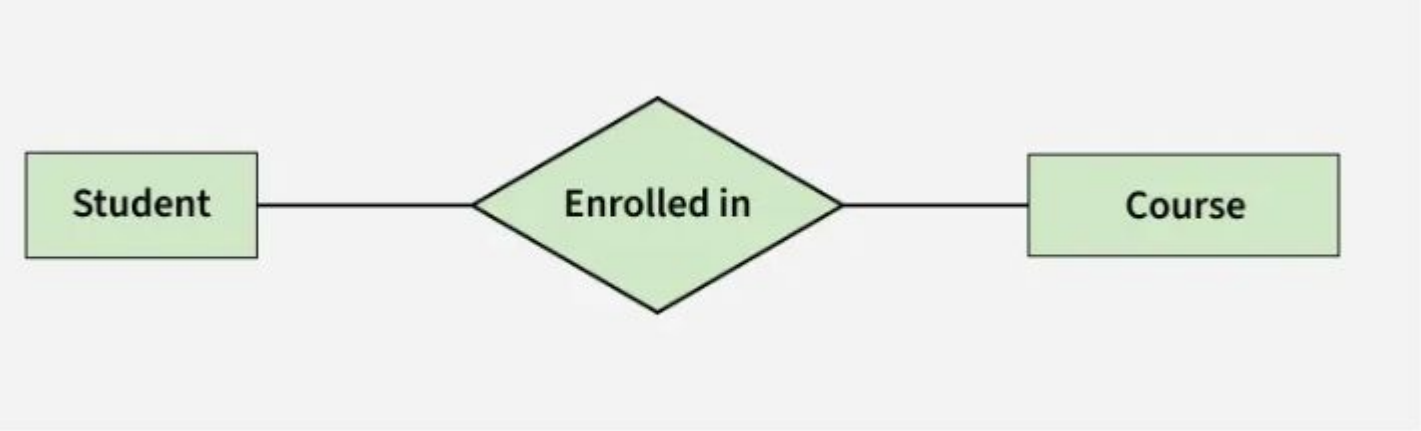
Degree of a Relationship Set

- The number of different entity sets participating in a relationship set is called the degree of a relationship set.
- 1. Unary/Recursive Relationship: When there is only ONE entity set participating in a relation, the relationship is called a unary relationship. For example, one person is married to only one person.



Unary Relationship

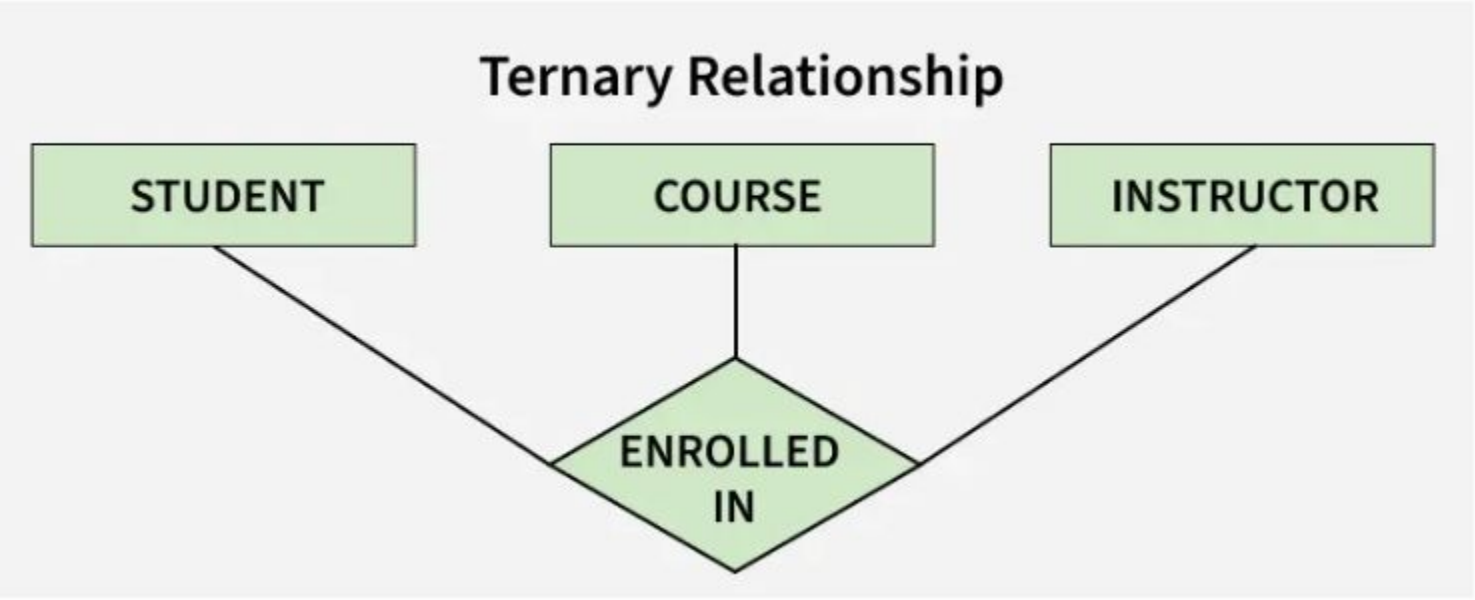
- Binary Relationship: When there are TWO entities set participating in a relationship, the relationship is called a binary relationship. For example, a Student is enrolled in a Course.



Binary Relationship

Ternary Relationship:

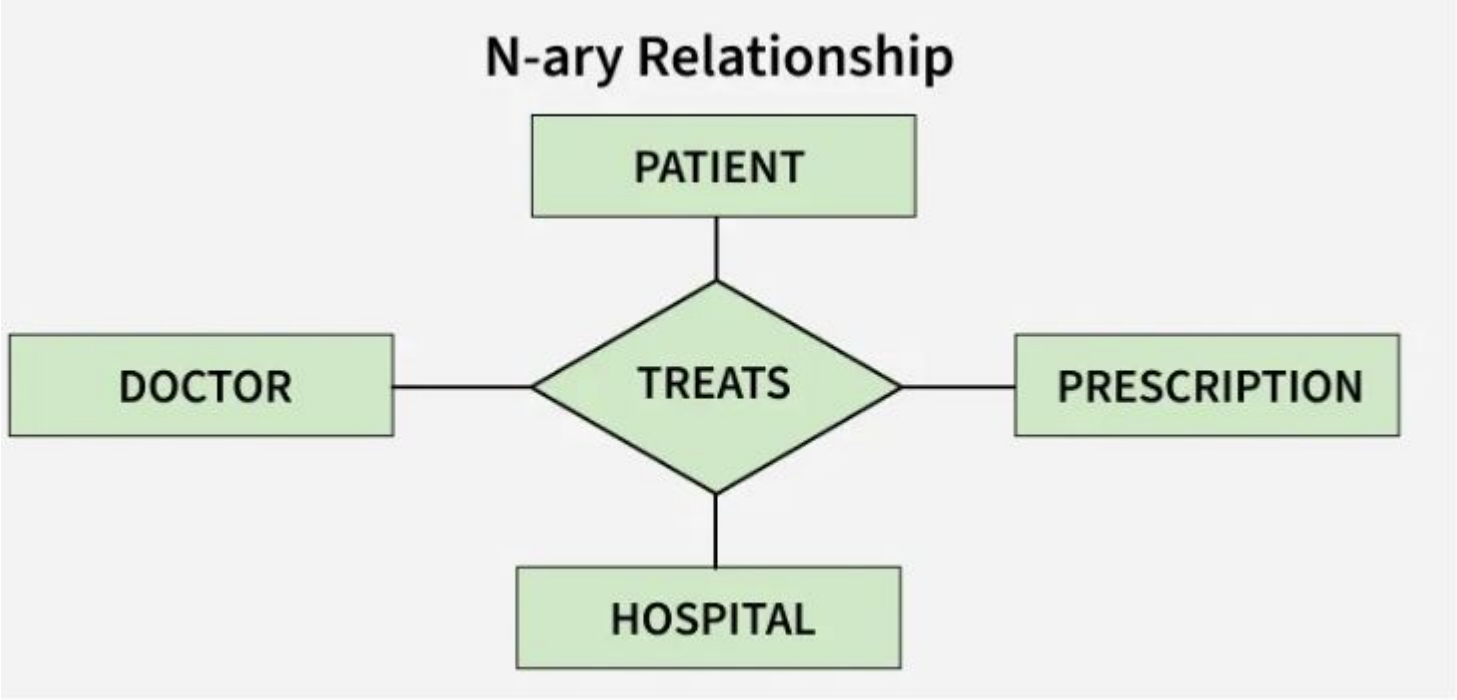
- When there are three entity sets participating in a relationship, the relationship is called a ternary relationship.



Ternary Relationship

N-ary Relationship:

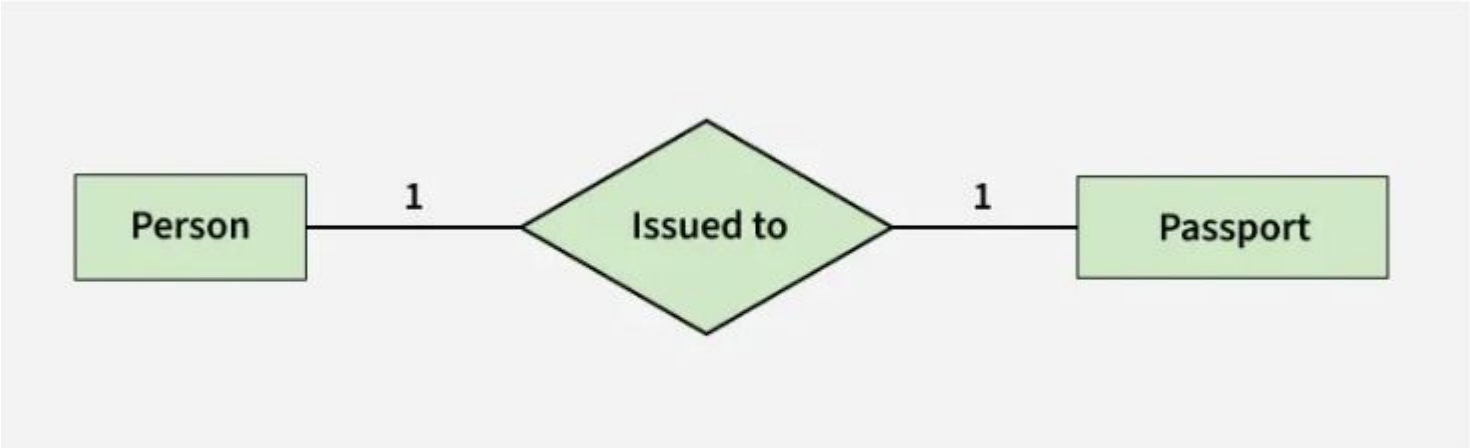
- When there are n entities set participating in a relationship, the relationship is called an n-ary relationship.



N-ary Relationship

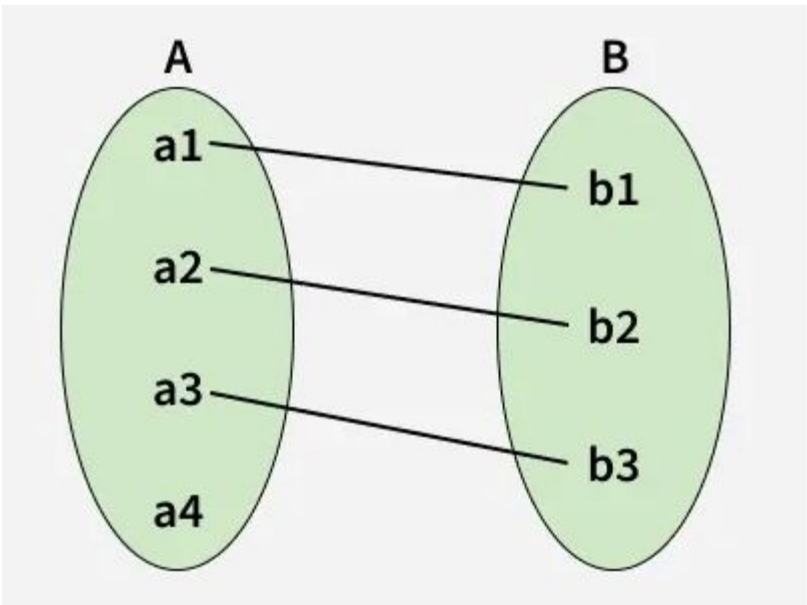
Cardinality in ER Model

- The maximum number of times an entity of an entity set participates in a relationship set is known as cardinality.Cardinality can be of different types:
- 1. One-to-One
- When each entity in each entity set can take part only once in the relationship, the cardinality is one-to-one. Let us assume that one person can be issued only one passport, and one passport is issued to only one person. So, the relationship will be One-to-One (1 : 1), meaning that each person has a single passport, and each passport belongs to a single person.



one to one cardinality

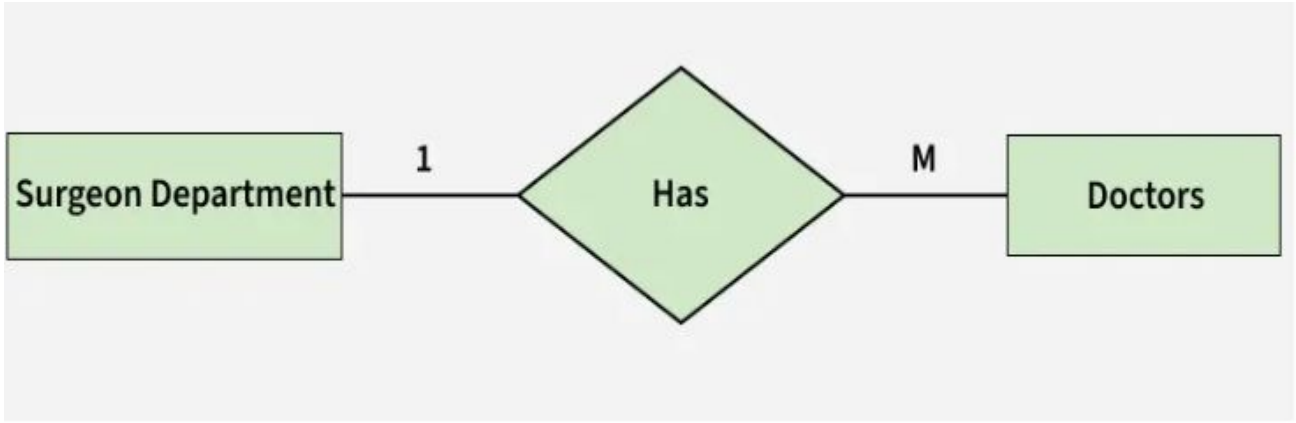
Using Sets, it can be represented as:



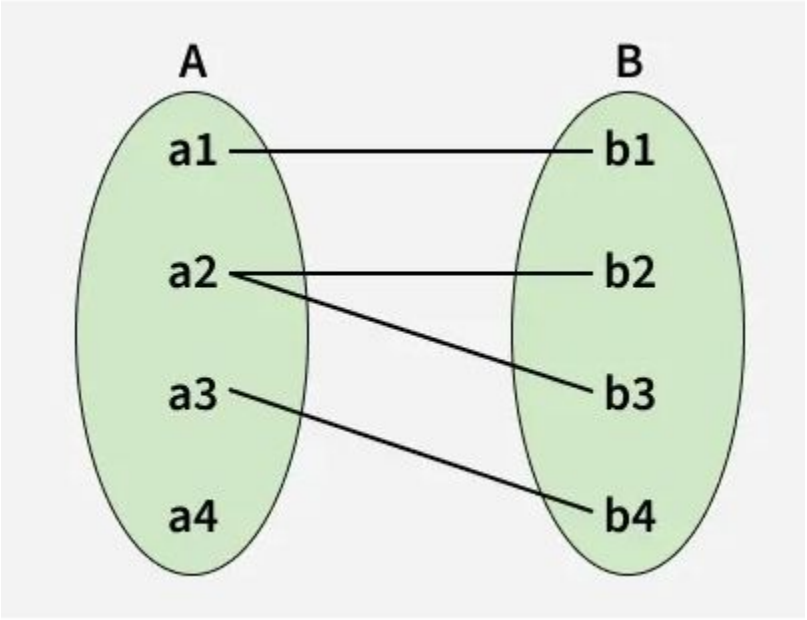
Set Representation of One-to-One

One-to-Many

- In a one-to-many relationship, one entity can be associated with multiple entities. For example, a single Surgeon Department can have many Doctors. Therefore, the cardinality of this relationship is 1 to M, meaning one department can have many doctors.



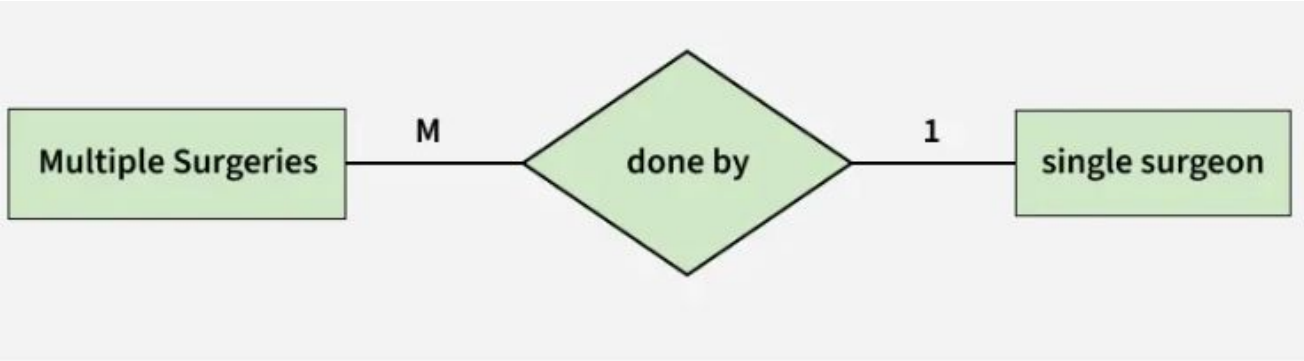
one to many cardinality



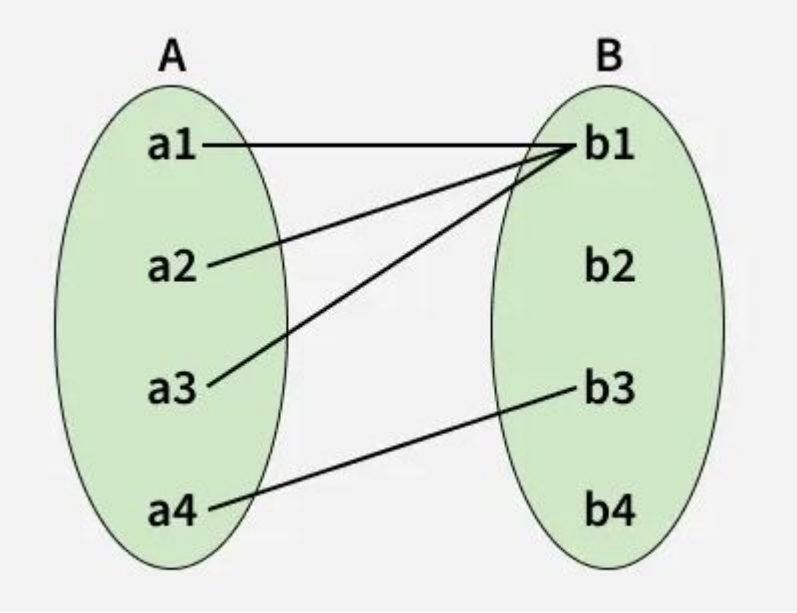
Set Representation of One-to-Many

Many-to-One

- When entities in one entity set can take part only once in the relationship set and entities in other entity sets can take part more than once in the relationship set, cardinality is many to one.
- Let us assume that multiple surgeries can be performed by one surgeon, but one surgery is performed by only one surgeon. So, the cardinality will be M to 1, meaning that many surgeries can be done by a single surgeon, but each surgery is done by only one surgeon.



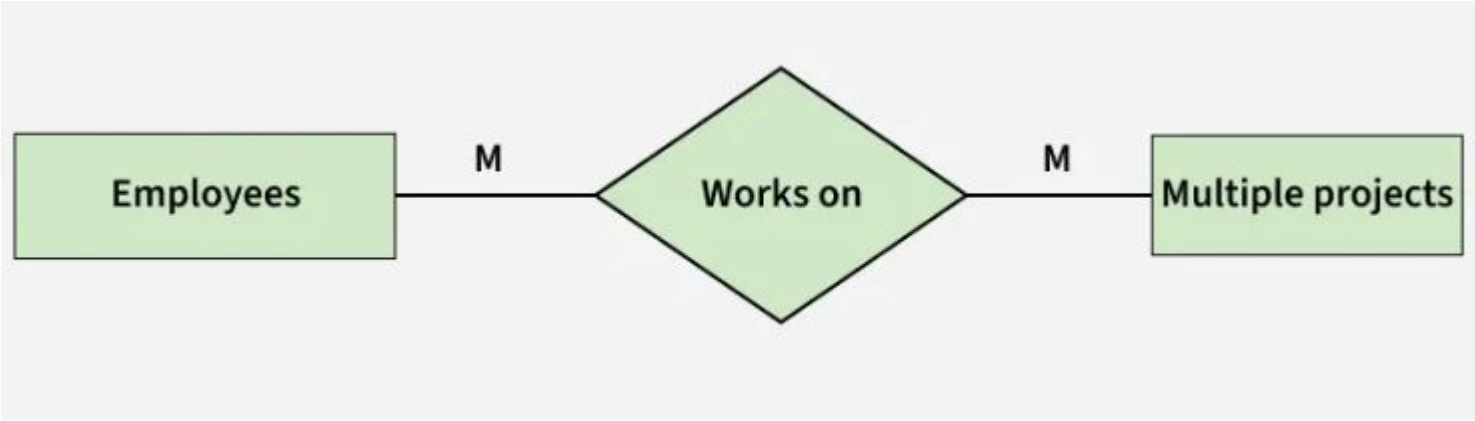
many to one cardinality



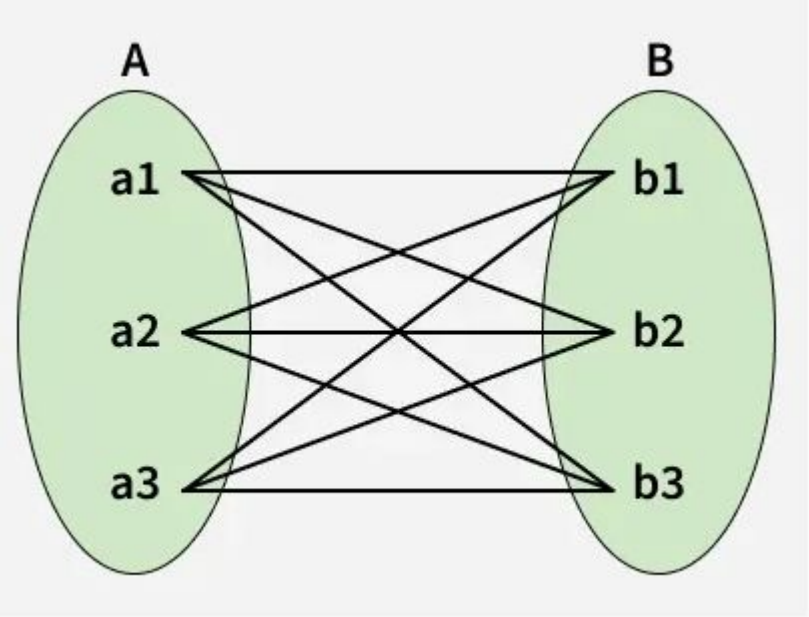
Set Representation of Many-to-One

Many-to-Many

- When entities in all entity sets can take part more than once in the relationship cardinality is many to many. Let us assume that an employee can work on multiple projects and each project can have multiple employees working on it. So, the relationship will be many-to-many (M:N), meaning that one employee may be associated with several projects, and one project may involve several employees.



many to many cardinality



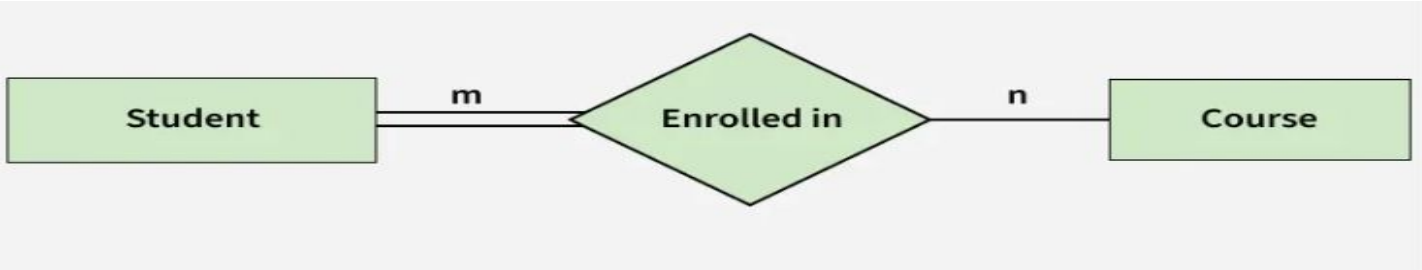
Many-to-Many Set Representation

Participation Constraint

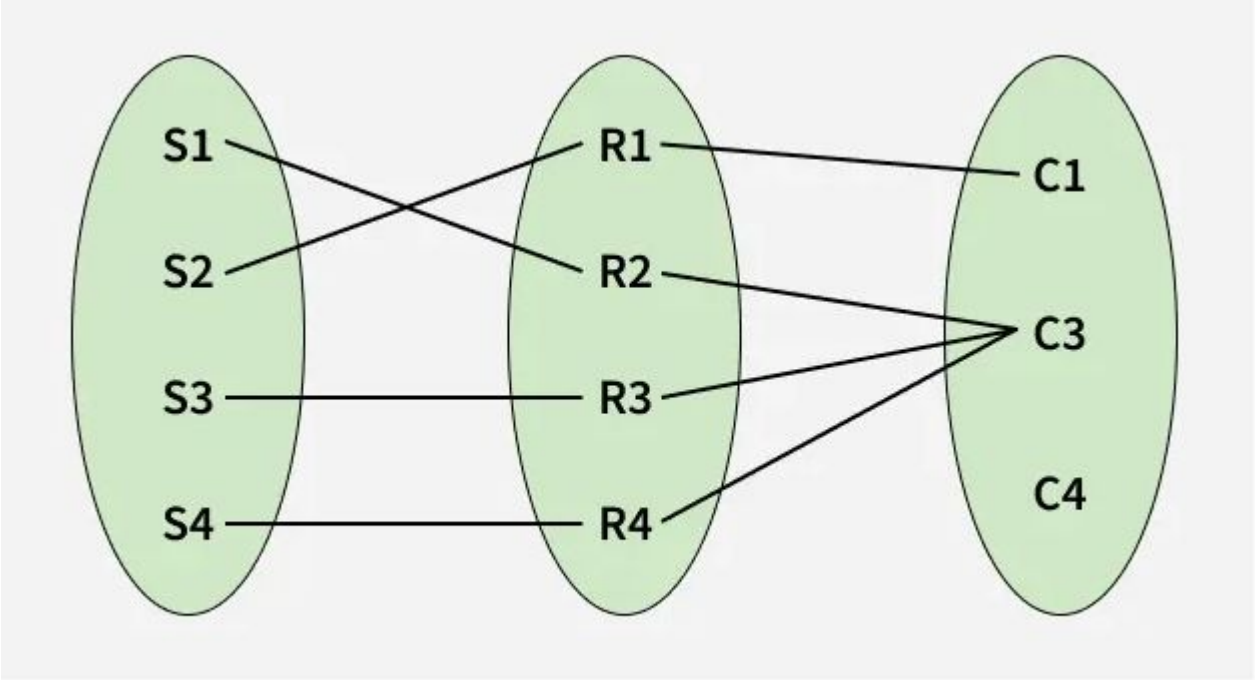
- Participation Constraint is applied to the entity participating in the relationship set.

- Total Participation: Each entity in the entity set must participate in the relationship. If each student must enroll in a course, the participation of students will be total. Total participation is shown by a double line in the ER diagram.
- Partial Participation: The entity in the entity set may or may NOT participate in the relationship. If some courses are not enrolled by any of the students, the participation in the course will be partial.

- The diagram depicts the 'Enrolled in' relationship set with Student Entity set having total participation and Course Entity set having partial participation.



Total Participation and Partial Participation



Set representation of Total Participation and Partial Participation

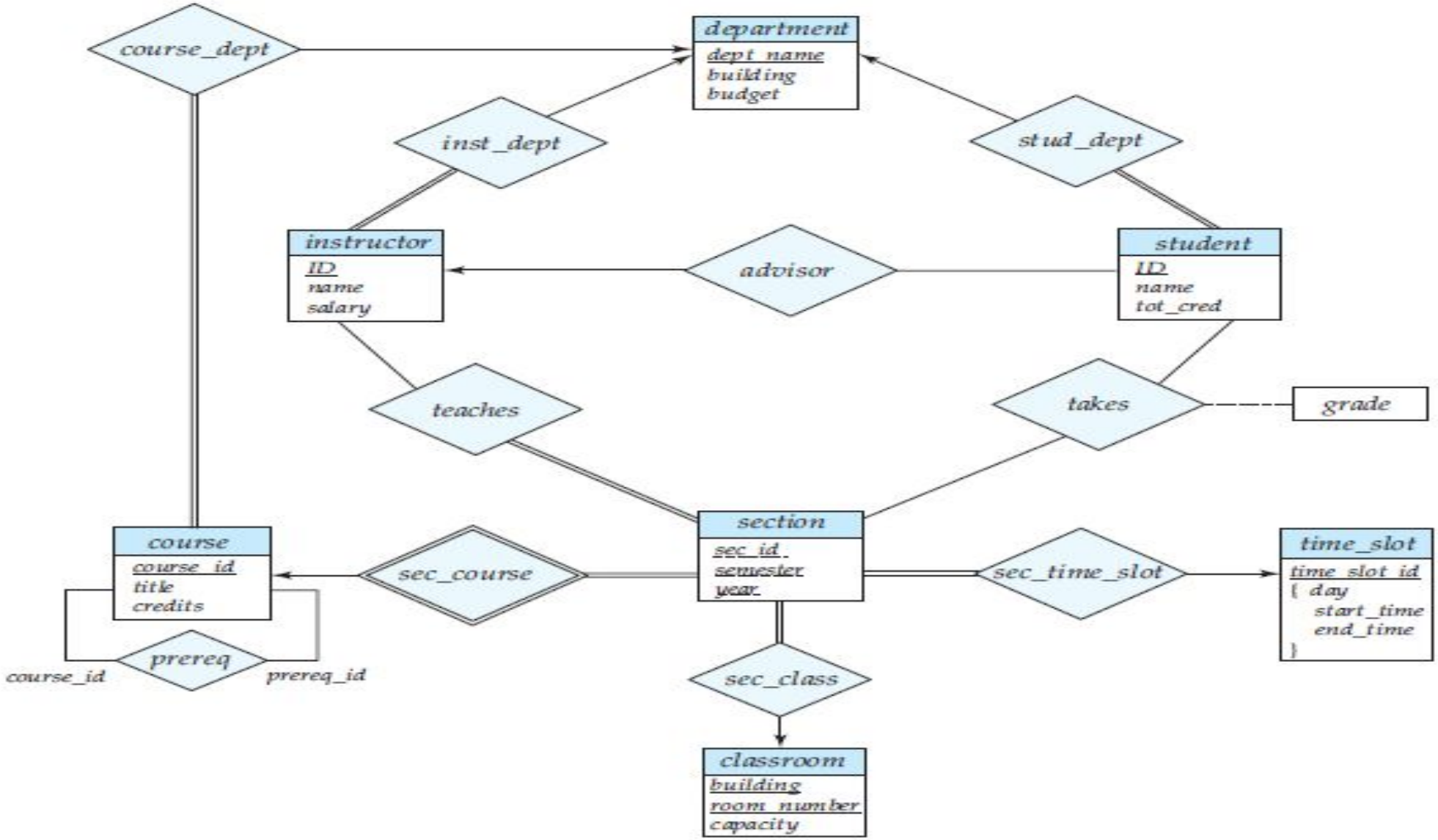


Figure 7.15 E-R diagram for a university enterprise.

Extended E-R Features

- CREATE DATABASE databasename;
  - CREATE DATABASE testDB;
- DROP DATABASE databasename;
- DROP DATABASE testDB;

```
CREATE TABLE table_name (
   column1 datatype,
    column2 datatype,
    column3 datatype,
   ....
);
```

```
CREATE TABLE Persons (
    PersonID int,
    LastName varchar(255),
    FirstName varchar(255),
    Address varchar(255),
    City varchar(255)
);
```

The LastName, FirstName, Address, and City columns are of type varchar and will hold characters, and the maximum length for these fields is 255 characters.

- DROP TABLE table_name;
- DROP TABLE Shippers;

- INSERT INTO table_name (column1, column2, column3, ...)

VALUES (value1, value2, value3, ...);

- INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country) VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway');

- Or

- INSERT INTO table_name

VALUES (value1, value2, value3, ...);

- INSERT INTO Customers (CustomerName, City, Country) VALUES ('Cardinal', 'Stavanger', 'Norway');

Some of The Most Important SQL Commands

- SELECT - extracts data from a database
- UPDATE - updates data in a database
- DELETE - deletes data from a database
- INSERT INTO - inserts new data into a database
- CREATE DATABASE - creates a new database
- ALTER DATABASE - modifies a database
- CREATE TABLE - creates a new table
- ALTER TABLE - modifies a table
- DROP TABLE - deletes a table
- CREATE INDEX - creates an index (search key)
- DROP INDEX - deletes an index

The SQL SELECT Statement

- SELECT *column1*, *column2, ...*
  FROM *table_name*;

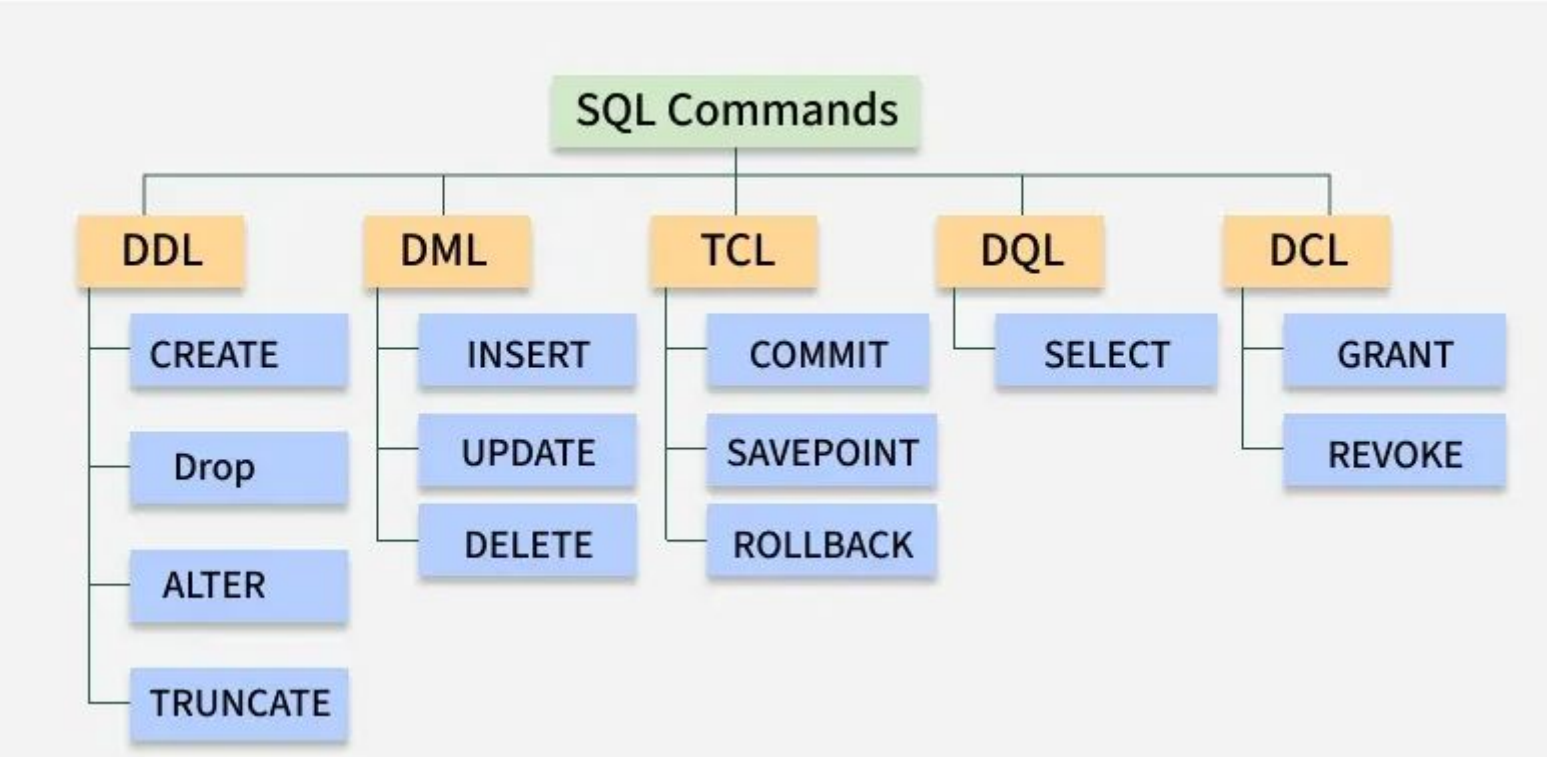
- SELECT CustomerName, City FROM Customers;

- Select ALL columns
- SELECT * FROM Customers;

- The SQL SELECT DISTINCT Statement
- The SELECT DISTINCT statement is used to return only distinct (different) values.
- SELECT DISTINCT *column1, column2, ...*
  FROM *table_name;*
- SELECT DISTINCT Country FROM Customers;

- SELECT Example Without DISTINCT
- If you omit the DISTINCT keyword, the SQL statement returns the "Country" value from all the records of the "Customers" table:
- SELECT Country FROM Customers;

DDL, DQL, DML, DCL and TCL Commands



Command	Description	Syntax
CREATE	Create database or its objects (table, index, function, views, store procedure and triggers)	CREATE TABLE table_name (column1 data_type, column2 data_type, ...);
DROP	Delete objects from the database	DROP TABLE table_name;
ALTER	Alter the structure of the database	ALTER TABLE table_name ADD COLUMN column_name data_type;
TRUNCATE	Remove all records from a table, including all spaces allocated for the records are removed	TRUNCATE TABLE table_name;
COMMENT	Add comments to the data dictionary	COMMENT ON TABLE table_name IS 'comment_text';
RENAME	Rename an object existing in the database	RENAME TABLE old_table_name TO new_table_name;

DQL - Data Query Language

Command	Description	Syntax
SELECT	It is used to retrieve data from the database	SELECT column1, column2, ...FROM table_name WHERE condition;
FROM	Indicates the **table(s)** from which to retrieve data.	SELECT column1 FROM table_name;
WHERE	Filters rows **before** any grouping or aggregation	SELECT column1 FROM table_name WHERE condition;

GROUP BY	Groups rows that have the same values in specified columns.	SELECT column1, AVG_FUNCTION(column2) FROM table_name GROUP BY column1;
HAVING	Filters the results of GROUP BY	SELECT column1, AVG_FUNCTION(column2) FROM table_name GROUP BY column1 HAVING condition;
DISTINCT	Removes **duplicate rows** from the result set	SELECT DISTINCT column1, column2, ... FROM table_name;
ORDER BY	Sorts the result set by one or more columns	SELECT column1 FROM table_name ORDER BY column1 [ASC | DESC];
LIMIT	By default, it sorts in **ascending order** unless specified as DESC	SELECT * FROM table_name LIMIT number;

DML - Data Manipulation Language

Command	Description	Syntax
INSERT	Insert data into a table	INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
UPDATE	Update existing data within a table	UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
DELETE	Delete records from a database table	DELETE FROM table_name WHERE condition;

DCL - Data Control Language

Command	Description	Syntax
GRANT	Assigns new privileges to a user account, allowing access to specific database objects, actions or functions.	GRANT privilege_type [(column_list)] ON [object_type] object_name TO user [WITH GRANT OPTION];
REVOKE	Removes previously granted privileges from a user account, taking away their access to certain database objects or actions.	REVOKE [GRANT OPTION FOR] privilege_type [(column_list)] ON [object_type] object_name FROM user [CASCADE];

TCL - Transaction Control Language

Command	Description	Syntax
BEGIN TRANSACTION	Starts a new transaction	BEGIN TRANSACTION [transaction_name];
COMMIT	Saves all changes made during the transaction	COMMIT;
ROLLBACK	Undoes all changes made during the transaction	ROLLBACK;
SAVEPOINT	Creates a savepoint within the current transaction	SAVEPOINT savepoint_name;

- Select all the different countries from the "Customers" table:
- Select all customers from Mexico:
- Select all customers with a CustomerID greater than 80:
- List all products order by price

The following operators can be used in the WHERE clause:

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. **Note:** In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

The SQL ORDER BY

- The ORDER BY keyword is used to sort the result-set in ascending or descending order.
- SELECT *column1*, *column2, ...*
  FROM *table_name*
  ORDER BY *column1, column2, ...* ASC|DESC;

1. Select all columns from the `customers` table.
2. Select only the `name` and `email` columns from the `customers` table.
3. Select all unique cities from the `customers` table.
4. Select the first 5 rows from the `products` table.
5. Select all columns from `orders` and rename the `order_date` column to `date_of_order`.
6. Select all products where the price is greater than 50.
7. Select all customers who live in 'London’.
8. Find all orders that were placed on or after '2023-01-01’.
9. Select all products that are out of stock (stock_quantity = 0).
10. Find all customers whose names start with 'J’.